

Resolução do Problema Treehouses (Kattis)

Árvore Geradora Mínima com
Conexões Pré-existentes

Integrantes: Paulo Afonso, Davi Martins, Isaac Coelho

Disciplina: Resolução de Problemas com Grafos

Orientador: Prof. Me Ricardo Carubbi

Trabalho Prático 1 - Unidade 3

O Problema Treehouses

Contexto e Requisitos do Sistema

Contexto

O objetivo é conectar casas na árvore em uma floresta tropical de forma que todas as casas e a área aberta (terra firme) sejam acessíveis entre si.

A conexão pode ser feita através de cabos aéreos retos. O custo de cada cabo é proporcional ao seu comprimento (distância euclidiana).

Objetivo

Adicionar a **menor quantidade total de cabo novo** para garantir a conectividade global do sistema.

Parâmetros de Entrada

Variável	Descrição
N	Número total de casas na árvore.
E	Primeiras E casas próximas à terra firme.
P	Número de cabos já instalados.

* As primeiras E casas são consideradas conectadas entre si e à terra firme com custo zero.

Modelagem como Grafo Ponderado

Conversão do Problema Real para Abstração Matemática

Vértices (V)

Cada uma das **N casas na árvore** é representada como um vértice no grafo (índices 0 a N-1).

Arestas (E) e Pesos

Pré-conexões (Custo 0):

- As primeiras **E casas** são unidas diretamente no DSU (equivalente a arestas de custo zero).
- Os **P cabos** já existentes entre casas são unidos diretamente no DSU (custo zero).

Arestas Candidatas (Peso Euclidiano):

- Entre qualquer par de casas i e j : $d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.



Estratégia Algorítmica: Kruskal + Union-Find

Abordagem para Árvore Geradora Mínima com Componentes Iniciais

Por que Kruskal?

O algoritmo de Kruskal é ideal para este problema pois trabalha com uma **lista global de arestas**, facilitando a integração de conexões pré-existentes.

Diferente do algoritmo de Prim, o Kruskal permite "unir" componentes distantes desde o início, o que se alinha perfeitamente com as **E casas da terra firme** e os **P cabos iniciais**.

O uso de **Union-Find (DSU)** com compressão de caminho e união por rank garante que as operações de verificação e união de componentes ocorram em tempo quase constante.

Passos Iniciais da Solução

- 1 Inicialização DSU:** Criar a estrutura DSU para os N vértices, cada um em seu próprio conjunto.
- 2 Pré-conexão (Custo Zero):** Realizar `dsu.union(i, i+1)` para as primeiras E casas e `dsu.union(a-1, b-1)` para os P cabos existentes.
- 3 Geração de Arestas:** Calcular a distância euclidiana entre todos os pares de casas (i, j) e armazenar como arestas candidatas.
- 4 Ordenação:** Ordenar todas as arestas candidatas pelo peso (distância) em ordem crescente.

Por que não Prim?

Análise Comparativa entre Kruskal e Prim para este Problema

Limitações do Prim

Crescimento Localizado

O Prim cresce a partir de um único nó inicial. Para lidar com componentes já conectados, ele exigiria a inserção de todos os nós pré-conectados na fila de prioridade desde o início.

Complexidade de Implementação

Gerenciar múltiplos componentes iniciais (E casas + P cabos) no Prim é menos intuitivo e exige uma estrutura de dados mais complexa para evitar ciclos entre componentes já existentes.

Vantagem Decisiva do Kruskal

Natureza Global

O Kruskal não se importa com a "vizinhança". Ele une quaisquer dois componentes no grafo, o que combina perfeitamente com a estrutura de

Union-Find (DSU)

.

Simplicidade com DSU

No Kruskal, as conexões iniciais são apenas operações `union` preliminares. O algoritmo "enxerga" o problema como a união de componentes já existentes, sem necessidade de lógica adicional.

Execução e Justificativa

Por que a estratégia de Kruskal garante a solução ótima?

Fluxo de Processamento

1. Pré-conexões no DSU

As primeiras E casas e os P cabos existentes são unidos no DSU, formando componentes iniciais de custo zero.

2. Ordenação e Seleção Gananciosa

Todas as arestas candidatas (distâncias euclidianas) são ordenadas. O Kruskal itera, adicionando a aresta mais barata que conecta dois componentes distintos ($dsu.find(u) \neq dsu.find(v)$).

3. União de Componentes

A operação $dsu.union(u, v)$ funde os conjuntos, reduzindo o número de componentes até que reste apenas um, minimizando o custo total.

Justificativa da Solução

Propriedade da MST: O algoritmo de Kruskal é comprovadamente ótimo para encontrar a árvore geradora de custo mínimo em qualquer grafo ponderado.

Adaptação ao Problema: Ao tratar conexões existentes como arestas de peso zero (via DSU), transformamos o problema de "conectar componentes" em uma busca padrão de MST.

Garantia de Conectividade: Como o grafo completo implícito é sempre conexo (podemos ligar qualquer par de casas), o algoritmo sempre encontrará um caminho de custo mínimo entre todos os pontos.

Análise de Complexidade

Eficiência Algorítmica e Uso de Recursos

Complexidade de Tempo

Etapa	Complexidade
Pré-conexões no DSU	$O(E + P * \alpha(N))$
Geração de Arestas	$O(N^2)$
Ordenação de Arestas	$O(N^2 \log N)$
Kruskal com DSU	$O(N^2 \alpha(N))$

Total: $O(N^2 \log N)$

Dominada pela geração e ordenação das N^2 arestas potenciais.

Complexidade de Espaço

Estrutura	Complexidade
Lista de Arestas	$O(N^2)$
Coordenadas (X, Y)	$O(N)$
Union-Find (Pais/Rank)	$O(N)$

Total: $O(N^2)$

Necessário para armazenar todas as conexões possíveis entre as casas.

Casos Especiais e Conclusão

Robustez da Solução e Considerações Finais

Casos Especiais

Sistema Já Conectado

Se as conexões iniciais (E e P) já unem todos os vértices, o algoritmo não seleciona arestas novas e retorna custo 0.0.

$N = 1$ ou $E = N$

Casos triviais onde a conectividade global já é garantida por definição, resultando em custo zero sem processamento de arestas.

Múltiplos Componentes Iniciais

O Union-Find agrupa eficientemente os componentes pré-existentes, tratando-os como "super-vértices" para a expansão da MST.

Conclusão

A modelagem do problema como uma **Árvore Geradora Mínima (MST)** com pré-processamento de componentes conectados provou ser a abordagem ideal.

O uso do algoritmo de **Kruskal** permitiu integrar naturalmente as restrições de custo zero, enquanto a estrutura **Union-Find** garantiu a eficiência necessária para os limites do problema.

A solução é robusta, tratando desde casos simples até florestas complexas com múltiplas conexões pré-existentes.

DATE	PROBLEM	JUDGEMENT	RUNTIME	LANGUAGE	TEST CASES
2026-05-24 22:28:13 ✓✓✓✓✓✓✓✓✓✓✓✓	Treehouses	✓ Accepted	1.58 s	Python 3	10/10

[Submitted files](#) [Submission history](#)

Files submitted

Mainfile: `treehouses.py`

treehouses.py

Download treehouses.py

```
1 import sys
2 import math
3
4 input = sys.stdin.readline
5
6
7 # -----
8 # Union-Find / DSU
9 # Compressão de caminho (recursiva) + união por rank
10 # -----
11
12 class DSU:
13     def __init__(self, n):
14         self.parent = list(range(n))
15         self.rank = [0] * n
16
17     def find(self, x):
18         # Compressão de caminho: faz cada nó apontar diretamente para a raiz
19         if self.parent[x] != x:
20             self.parent[x] = self.find(self.parent[x])
21         return self.parent[x]
22
23     def union(self, x, y):
24         rx, ry = self.find(x), self.find(y)
25
26         if rx == ry:
27             return False # já estão no mesmo componente - aresta descartada
28
29         # União por rank: árvore menor fica abaixo da maior
30         if self.rank[rx] < self.rank[ry]:
31             rx, ry = ry, rx
32
33         self.parent[ry] = rx
34
35         if self.rank[rx] == self.rank[ry]:
36             self.rank[rx] += 1
37
38         return True
39
40
41 # -----
42 # Função principal
43 # -----
44
45 def solve():
46     n, e, p = map(int, input().split())
47
48     # Lê as coordenadas dos treehouses (0-indexado internamente)
49     coords = []
```